Dunaújvárosi Egyetem Bánki Donát Technikum

Projekt feladat dokumentáció

Tartalom

Az ötlet rövid leírása: 2
Hozzávalók és költségvetés 2
Működési elv 2
Kapcsolási rajz 3
Arduino IDE Kód 4
Fejlesztési lehetőségek 7
Önreflexió 8

Tantárgy neve: IoT

Projekt tervező:

Barkóczi Rajmund Soma

Projekt címe: Okos növénygondozó

Osztály: 13.B

Dátum: 2025.12.09.

Az ötlet rövid leírása:

A projekt egy olyan intelligens környezetfigyelő eszköz, amely segít az optimális életfeltételek biztosításában szobanövények számára. A cél, hogy a felhasználó **távolról, interneten keresztül** is valós idejű adatokat kapjon a növény állapotáról, és azonnali riasztást küldjön, ha beavatkozás (öntözés) szükséges

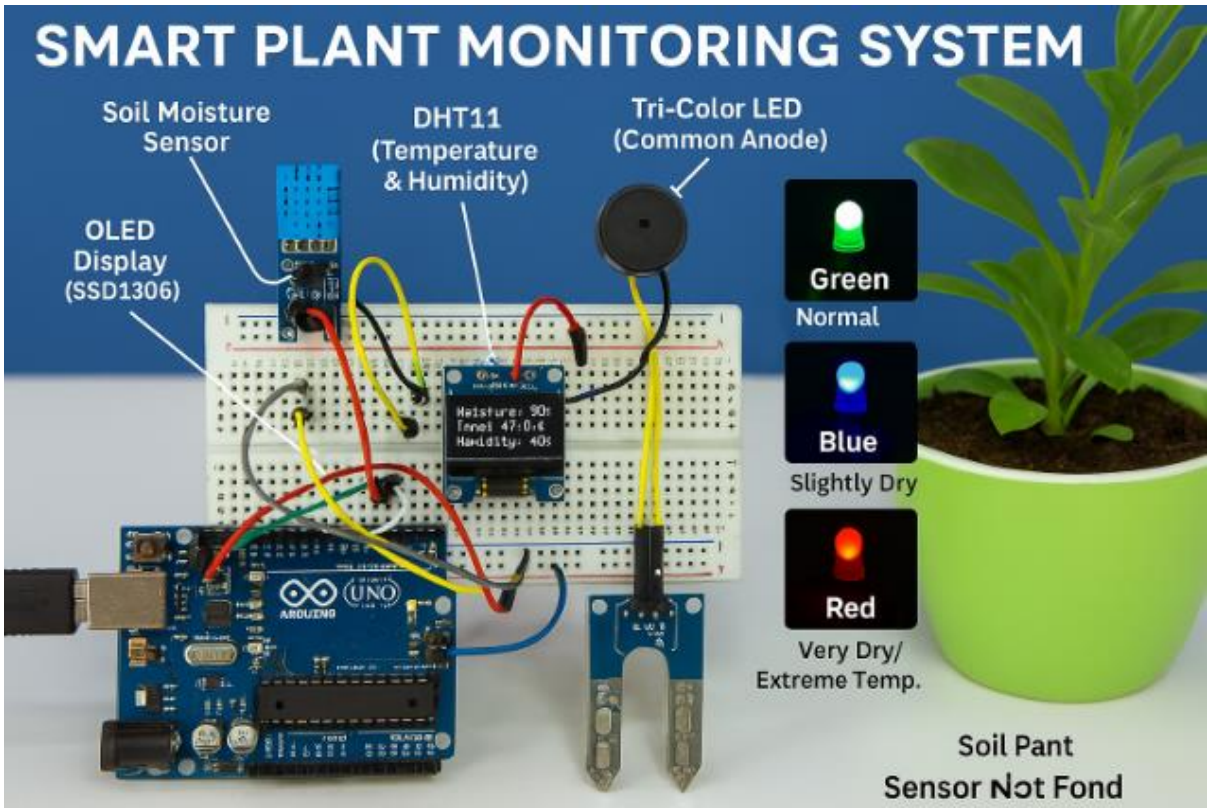
Hozzávalók és költségvetés

1	Alkatrész	Mennyiség	Becsült ár (bruttó)	Megjegyzés
2	NodeMCU V3 (ESP8266)	1 db	2 500 - 3 000 Ft	Az "agy", ezen van a Wi-Fi.
3	Kapacitív talajnedvesség mérő	1 db	800 - 1 200 Ft	Érdemes a kapacitívat venni (fekete), mert nem korrodál el úgy, mint az olcsó tüskés verzió.
4	DHT11 hő- és páramérő	1 db	1 000 - 1 500 Ft	Digitális szenzor, egyszerű bekötni.
5	Breadboard (830 pontos)	1 db	1 200 - 1 800 Ft	Ezen tudod összeállítani forrasztás nélkül.
6	Jumper kábel készlet (apa-apa, apa-anya)	1 csomag	800 - 1 200 Ft	Az alkatrészek összekötéséhez.
7	Micro USB kábel	1 db	800 - 1 500 Ft	Programozáshoz és áramellátáshoz (ha nincs otthon).
8	ÖSSZESEN:		~7 100 - 10 200 Ft	

Működési elv

Az okos szobanövény projekt működési elve egy folyamatos adatátviteli láncra épül, amely a fizikai környezet érzékelésével kezdődik, ahol a talajnedvesség-szenzor a föld elektromos ellenállását vagy kapacitását méri, miközben a DHT11 modul a levegő hőmérsékletét és páratartalmát figyeli. Ezeket a gyűjtött analóg és digitális jeleket az ESP8266 mikrokontroller fogadja be, amely az analóg-digitális átalakítója segítségével értelmezhető számadatokká formálja a nyers feszültségszinteket, majd a programozott logika alapján százalékos értékekké skálázza azokat. Miután az adatok előálltak, a vezérlő a beépített Wi-Fi modulján keresztül titkosított TCP/IP kapcsolaton keresztül csatlakozik a helyi hálózathoz, és HTTP vagy MQTT protokoll használatával továbbítja az információkat egy felhőalapú IoT szerverre. A távoli szerver az érkező adatcsomagokat feldolgozza, adatbázisban rögzíti a későbbi grafikonos elemzésekhez, és amennyiben a mért értékek átlépnek egy előre meghatározott kritikus küszöböt, automatikus utasítást küld a kapcsolt mobilalkalmazásnak. A folyamat végén a felhasználó a telefonján vagy egy webes felületen élőben látja a növény állapotát tükröző mutatókat, és push-értesítés formájában azonnali riasztást kap, ha a talaj kiszáradt, így az IoT technológia révén a fizikai állapotváltozásból digitális információ, majd távoli felhasználói interakció válik.

Kapcsolási rajz



Arduino IDE Kód

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// OLED Configuration (I2C: SDA=A4, SCL=A5)
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Pin Definitions
#define SOIL_MOISTURE_PIN A0   // Soil Moisture Sensor
#define DHT_PIN 2              // DHT11 Sensor
#define DHT_TYPE DHT11         // DHT11 Sensor Type
#define BUZZER_PIN 6           // Buzzer

// Tri-Color LED (Common Anode)
#define RED_LED 3              // CO1 - Red
#define GREEN_LED 4            // CO2 - Green
#define BLUE_LED 5             // CO3 - Blue

// Initialize DHT Sensor
DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
```

```
    pinMode(SOIL_MOISTURE_PIN, INPUT);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(RED_LED, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);
    pinMode(BLUE_LED, OUTPUT);

    // Initialize OLED Display
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("OLED Display not found!");
        while (1);
    }
    display.clearDisplay();
    Serial.println("System Initialized!");
}

// Function to set LED color
void setColor(bool red, bool green, bool blue) {
    digitalWrite(RED_LED, red ? LOW : HIGH);
    digitalWrite(GREEN_LED, green ? LOW : HIGH);
    digitalWrite(BLUE_LED, blue ? LOW : HIGH);
}

void loop() {
    // Read Raw Soil Moisture Value
    int moistureValue = analogRead(SOIL_MOISTURE_PIN);
    Serial.print("Raw Moisture Value: ");
    Serial.println(moistureValue);
```

```
// Adjust Mapping Based on Real Readings
int moisturePercentage = map(moistureValue, 300, 900, 0, 100);
moisturePercentage = constrain(moisturePercentage, 0, 100);

// Read Temperature & Humidity
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// Check if DHT11 gives valid data
if (isnan(temperature) || isnan(humidity)) {
    Serial.println("DHT11 Sensor Not Found!");
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);
    display.setCursor(10, 20);
    display.print("DHT11 Not Found!");
    display.display();
    delay(2000);
    return;
}

// Print values in Serial Monitor
Serial.print("Moisture: "); Serial.print(moisturePercentage); Serial.println("%");
Serial.print("Temperature: "); Serial.print(temperature); Serial.println(" °C");
Serial.print("Humidity: "); Serial.print(humidity); Serial.println(" %");

// Display Sensor Data on OLED
display.clearDisplay();
display.setTextSize(1);
```

```
  display.setTextColor(WHITE);
  display.setCursor(10, 10);
  display.print("Moisture: "); display.print(moisturePercentage); display.println("%");
  display.setCursor(10, 30);
  display.print("Temp: "); display.print(temperature); display.println("C");
  display.setCursor(10, 50);
  display.print("Humidity: "); display.print(humidity); display.println("%");
  display.display();

  // LED & Buzzer Alerts
  if (moisturePercentage < 30 || temperature > 30.0 || temperature < 20.0) {
    setColor(1, 0, 0);  //  Red (Critical Condition)
    digitalWrite(BUZZER_PIN, HIGH);
  } else if (moisturePercentage < 40) {
    setColor(0, 0, 1);  //  Blue (Warning Condition)
    digitalWrite(BUZZER_PIN, LOW);
  } else {
    setColor(0, 1, 0);  //  Green (Normal Condition)
    digitalWrite(BUZZER_PIN, LOW);
  }

  delay(2000);  // Refresh every 2 seconds
}
```

Fejlesztési lehetőségek

- Automatizált öntözés: Egy 5V-os vízpumpa és egy relémodul hozzáadásával a rendszer nemcsak jelezni tud, hanem önműködően meg is öntözi a növényt, ha a talaj kiszárad.

- Napelemes tápellátás: Egy kisméretű napelemmel és egy Li-ion akkumulátorral (például 18650-es cellával) az eszköz teljesen önfenntartóvá és hordozhatóvá tehető.

- Energiahatékonyság (Deep Sleep): A szoftver módosításával a kontroller két mérés között „mélyalvás” üzemmódba léphet, így az áramfelvétele töredékére csökken, ami elengedhetetlen az akkumulátoros IoT eszközöknél.

- Helyi kijelző: Egy apró, 0.96 colos OLED kijelző beépítésével a legfontosabb adatok (hőmérséklet, nedvesség) a helyszínen, telefon nélkül is leolvashatók.

- Fényintenzitás mérése: Egy fotorezisztor (LDR) segítségével figyelhető, hogy a növény kap-e elég természetes fényt, vagy szükség van-e mesterséges megvilágításra.

- Többcsatornás figyelés: A rendszer bővíthető úgy, hogy egyszerre több növény adatait gyűjtse össze, vagy akár egy egész üvegházat felügyeljen.

- Okosotthon integráció: Az adatok továbbíthatók saját központi szerverre (például Home Assistant vagy OpenHAB), így a növénygondozás a lakás többi okoseszközével is összehangolható.

Önreflexió

Az Internet of Things (IoT) tantárgy keretében megvalósított projekt egy komplex mérnöki esettanulmányként értelmezhető, amely a fizikai rétegtől egészen az alkalmazási rétegig lefedi a teljes technológiai felépítést. A kurzus célkitűzéseivel összhangban a fejlesztés nem egy önálló áramkör elkészítésére korlátozódott, hanem egy vertikálisan integrált rendszer megtervezésére és megvalósítására irányult. Ennek során a beágyazott rendszerek programozása, a szenzoradatok kezelése, a vezeték nélküli kommunikáció, valamint a felhőalapú adatfeldolgozás egyaránt szerepet kaptak.

A projekt kiváló lehetőséget biztosított a tantárgy során elsajátított elméleti ismeretek gyakorlati alkalmazására. Az olyan alapfogalmak, mint az analóg jelek digitalizálása, a mintavételezési elvek vagy az adatok szerveroldali strukturálása, egy valós élettani folyamat – a növény állapotának monitorozása – révén nyertek gyakorlati jelentőséget.

A hálózati réteg kialakítása során kiemelt figyelmet kapott az adatátvitel megbízhatósága és a biztonságos kommunikáció. Az ESP8266 mikrokontroller kliensként történő hitelesítése, valamint a TCP/IP protokoll alkalmazása alapvető hálózati ismereteket és gyakorlati kompetenciákat igényelt. Az alkalmazási réteg szintjén a projekt hangsúlyozza az ember–gép interfész (HMI) szerepét, ahol a nyers szenzoradatok értelmezhető, vizuális formában jelennek meg, lehetővé téve a távoli megfigyelést és az esetleges beavatkozást.

Összességében az okos szobanövény projekt jól szemlélteti az IoT tantárgy multidiszciplináris jellegét, mivel egyaránt megköveteli a hardveres interfészek kezelését, az alacsony szintű C++ programozást és a modern webes technológiák alkalmazását. Ezek az

elemek egy funkcionális, bővíthető és skálázható rendszerarchitektúrában egyesülnek, amely hatékonyan tükrözi a tantárgy gyakorlatorientált szemléletét.